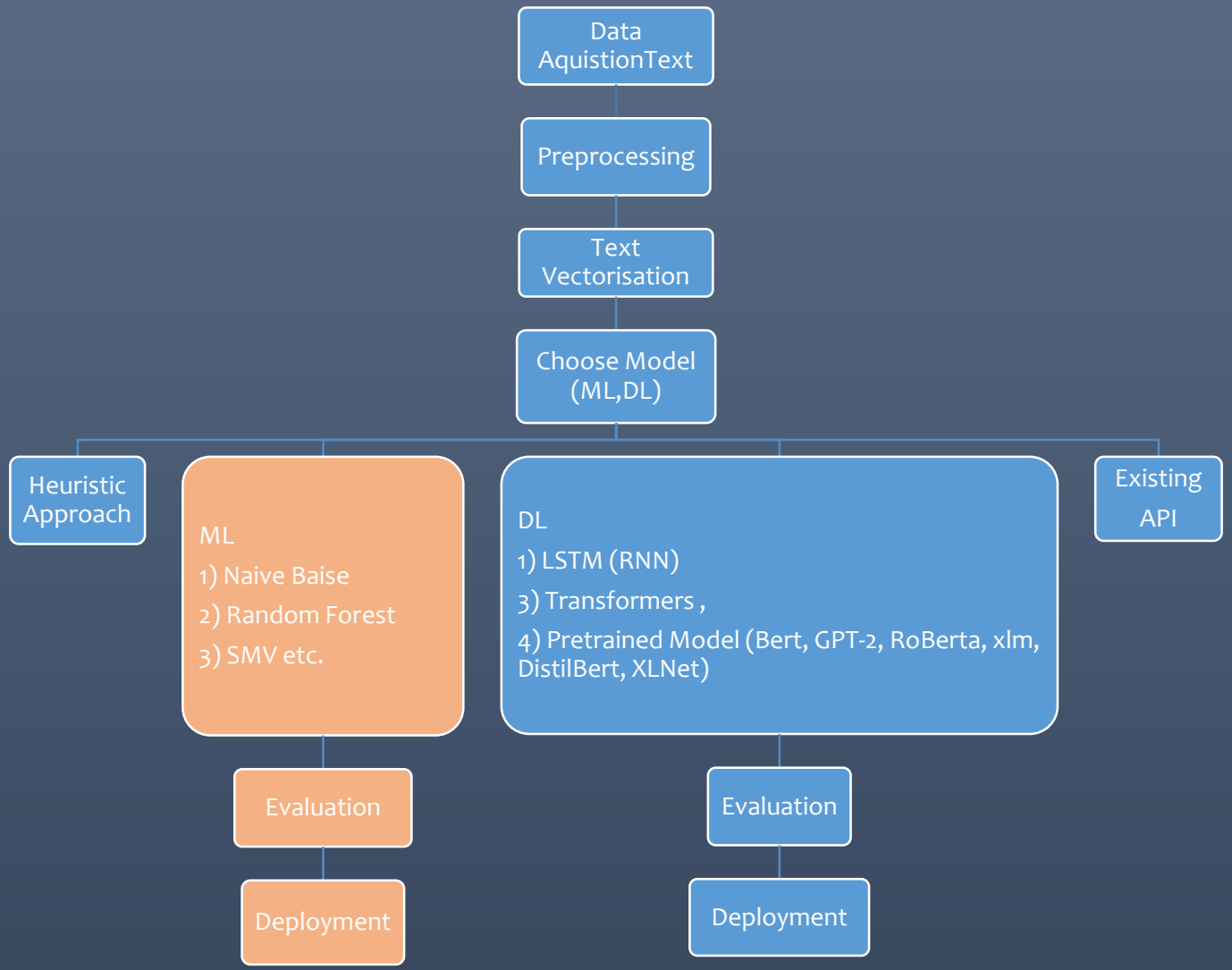

Data AquistionText
Preprocessing
Text Vectorisation
Choose Model (ML,DL)
Heuristic Approach
ML
1) Naive Baise
2) Random Forest
3) SMV etc.
DL
1) LSTM (RNN)
3) Transformers ,
4) Pretrained Model (Bert, GPT-2, RoBerta, xlm, DistilBert, XLNet)
Existing API
Evaluation
Deployment
Evaluation
Deployment

1) Data Acquisition
2) Data Augmentation
3) Text Preparation
    a. Text Cleaning:
    b. Pre-processing
4) Text Representation (Feature Engineering/Text Vectorisation/ Word Embedding's):
5) Model Building
6) Deployment

Text Preparation:

There are total three ways of text preparation

a) **Text Cleaning:**
b) **Pre-processing**

Text Cleaning:

Removing of unnecessary or irrelevant information that could hinder analysis or model performance.

- Lower casing
- Html Tag Cleaning
- Remove links
- Remove punctuation
- Chat Word treatment
- Spelling checker
- Stop-word removal
- Emoji Cleaning
  - REMOVE Emoji
  - Replace with meaning of Emoji

Pre-processing

Pre-processing in NLP involves several techniques that transform raw text into a structured format suitable for analysis or modelling. These techniques include:

- **Tokenization**
- **Stemming:**
- **Lemmatization:**
- **Part-of-Speech (POS) Tagging:**

Tokenization: Tokenization breaks text into smaller units, such as sentences (sentence tokenization) or words (word tokenization), which are essential for subsequent analysis.

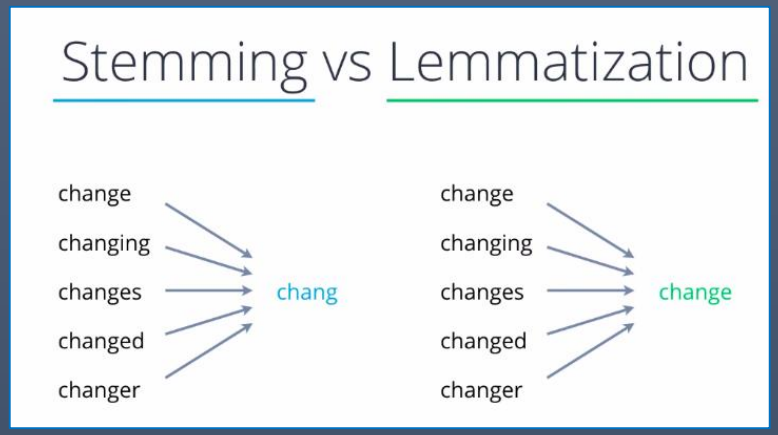


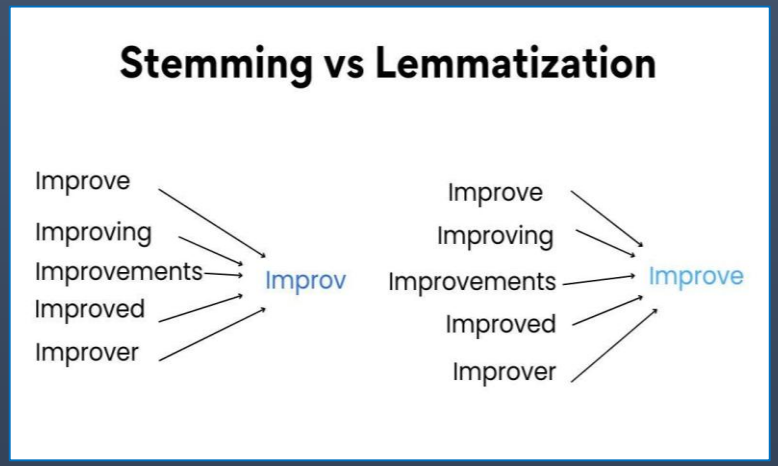
Stemming:

Stemming reduces words to their root form, even if the root itself may not be a valid word, helping to capture variations of words. Stemming is faster as compared to Lemmatization.

Lemmatization:

Lemmatization is similar to stemming but ensures words are transformed to their canonical forms based on context and meaning, resulting in valid words. It is little slow as compared to Stemming



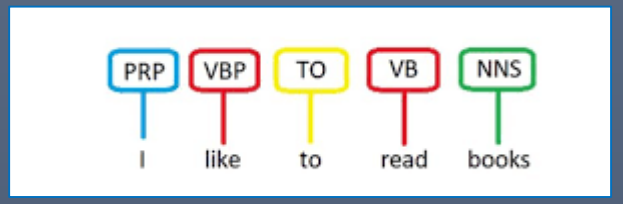


Original Word	Stemmed Form	Lemmatized Form
Playing	Play	play
Plays		
Played		
Cats	cat	cat
Happier	happi	happy
Happiest		
Easily	easili	easily
Houses	hous	house
Crying	cri	cry
Cried		
Strangely	strang	strangely
Becoming	becom	become

Part-of-Speech (POS) Tagging: POS tagging assigns grammatical categories (like noun, verb, adjective) to words in a sentence, crucial for syntactic analysis and understanding sentence structure.

Sentence: "She walks quickly to the store to buy groceries."

Word	POS Tag
She	PRP
walks	VBZ
quickly	RB
To	TO
the	DT
store	NN
to	TO
buy	VB
groceries	NNS
.	.



PRP : Personal pronoun (She)
VBZ : Verb, 3rd person singular present (walks)
RB : Adverb (quickly)
TO : to (preposition)
DT : Determiner (the)
NN : Noun, singular or mass (store)
VB : Verb, base form (buy)
NNS : Noun, plural (groceries)
. : Punctuation mark (period)

Feature Engineering || Text Representation || Text Vectorisation || Word Embedding

Feature engineering in natural language processing (NLP) involves transforming raw text data into numerical features that can be effectively used by machine learning models.

Here are some common feature engineering techniques used in NLP:

Types

1) Frequency-based
    - **OHE**
    - **Bow**
    - **N-grams**
    - **Tfidf,**
    - GloVe
    - Matrix
    - fastText

2) Prediction based: this is done based on prediction of the algorithms prediction
    - word2vec
        A) Continuous Bag of Words (CBOW):
        B) Skip-gram:

One-Hot Encoding:

	Example of comments
D1	People watch YouTuber
D2	YouTuber watch YouTuber
D3	People write Comment
D4	YouTuber write Comment

	People	watch	YouTuber	write	Comment
D1	1	0	0	0	0
	0	1	0	0	0
	0	0	1	0	0

	People	watch	YouTuber	write	Comment
D2	0	0	1	0	0
	0	1	0	0	0

	People	watch	YouTuber	write	Comment
D3	1	0	0	0	0
	0	0	0	1	0
	0	0	0	0	1

	People	watch	YouTuber	write	Comment
D4	0	0	1	0	0
	0	0	0	1	0
	0	0	0	0	1

D1	**[[1,0,0,0,0][0,1,0,0,0][0,0,1,0,0]**
D2	[[0,0,1,0,0][0,1,0,0,0]]
D3	[[1,0,0,0,0][0,0,0,1,0][0,0,0,0,1]]
D4	[[0,0,1,0,0][0,0,0,1,0][0,0,0,0,1]]

Bag of Words (BoW):

BoW represents text data as a matrix where rows correspond to documents and columns correspond to unique words (or n-grams) in the corpus. Each cell in the matrix represents the frequency of the word's occurrence in the corresponding document. BoW disregards the order of words in the document and only considers their frequencies.

	Example of comments
D1	People watch YouTuber
D2	YouTuber watch YouTuber
D3	People write Comment
D4	YouTuber write Comment

	People	watch	YouTuber	write	Comment
D1	1	1	1	0	0
D2	0	1	2	0	0
D3	1	0	0	1	1
D4	0	0	1	1	1

N-grams:

1) **One-gram**
2) **Bi-gram**
3) **Tri-gram**

N-grams are contiguous sequences of n words from a given text. They capture local context and dependencies between adjacent words. Both unigrams (single words) and higher-order n-grams can be used as features in NLP tasks.

	Example of comments
D1	People watch YouTuber
D2	YouTuber watch YouTuber
D3	People write Comment
D4	YouTuber write Comment

	People	watch	YouTuber	write	Comment
D1	1	1	1	-	-
D2	-	1	1	-	-
D3	1	-	-	1	1
D4	1	-	-	1	1

One-gram

	People watch	watch YouTuber	YouTuber watch	People write	write Comment	YouTuber write
D1	1	1	0	0	0	0
D2	0	1	1	0	0	0
D3	0	0	0	1	1	0
D4	0	0	0	0	1	1

Bi-gram

	People watch YouTuber	YouTuber watch YouTuber	People write Comment	YouTuber write Comment
D1	1	0	0	0
D2	0	1	0	0
D3	0	0	1	0
D4	0	0	0	1

Tri-gram

TF x IDF (Term Frequency-Inverse Document Frequency):

- TF-IDF calculates the importance of a word in a document relative to its importance in the entire corpus.

	Example of comments
D1	People watch YouTuber
D2	YouTuber watch YouTuber
D3	People write Comment
D4	YouTuber write Comment

$$TF = Inverse\ Document\ Frequency$$
$$IFD = Term\ Frequency$$

$$TF(i,j) = \frac{Term\ i\ Frequency\ in\ document\ j}{Total\ words\ in\ Documents\ j}$$

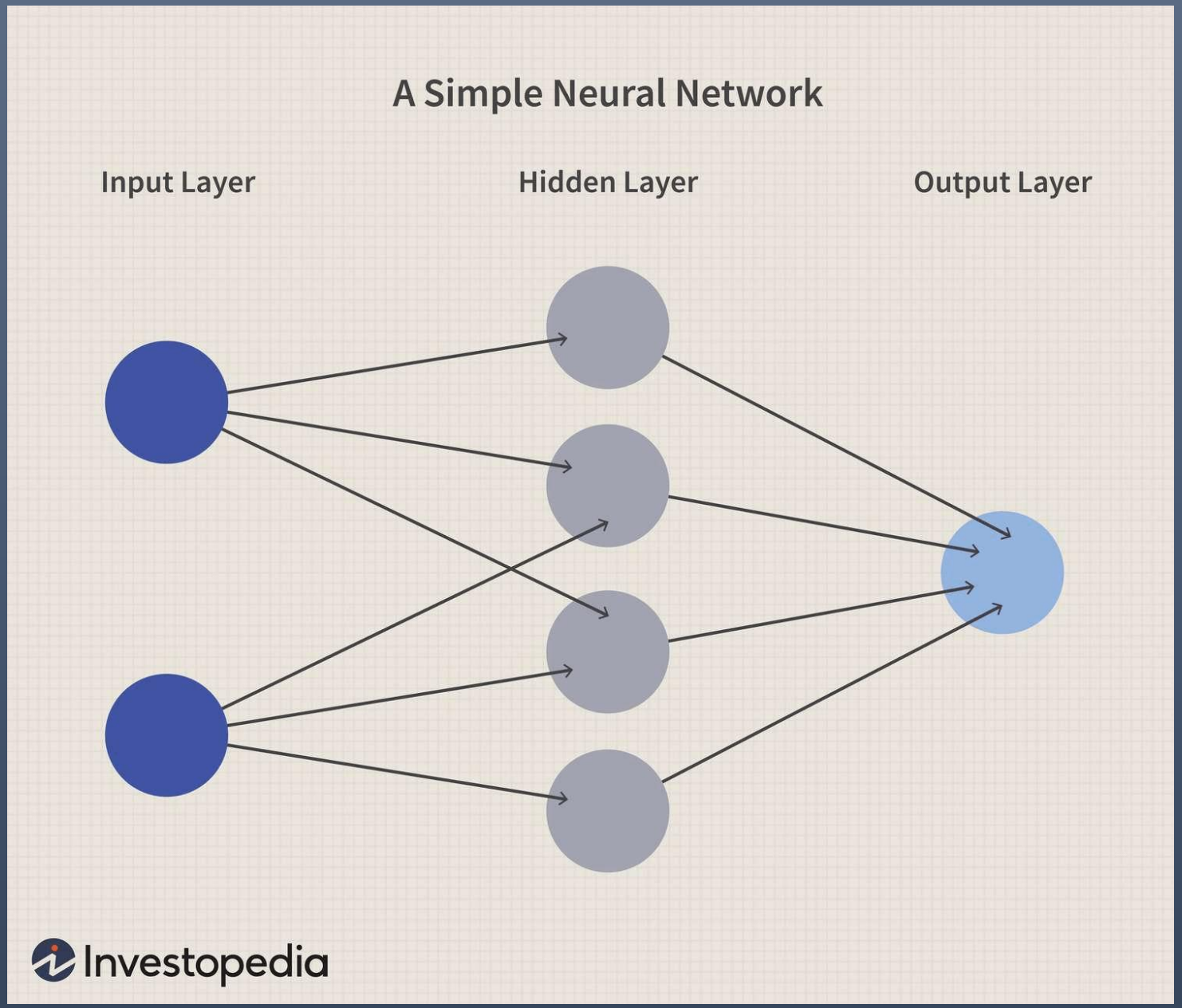
$$IDF(i) = \log\left(\frac{Total\ Documents}{Documents\ with\ term\ i}\right)$$

$$TF\ \&\ IFD\ Score = TF * IDF$$

		People	watch	YouTuber	write	Comment
TF	D1	0.33	0.33	0.33	-	-
	D2	-	0.33	0.67	-	-
	D3	0.33	-	-	0.33	0.33
	D4	0.33	-	-	0.33	0.33

		People	watch	YouTuber	write	Comment
IDF	For all Docs	1.12	1.12	1.30	1.12	1.12

		People	watch	YouTuber	write	Comment
TF x IDF	D1	0.37	0.37	0.43	-	-
	D2	-	0.37	0.87	-	-
	D3	0.37	-	-	0.37	0.37
	D4	0.37	-	-	0.37	0.37


A Simple Neural Network
Input Layer
Hidden Layer
Output Layer
Investopedia

Prediction based

- NNLM
- FastText
- Word2vec:

    This is done based on prediction using neural network.

Word2Vec

is a technique in natural language processing (NLP) that creates dense vector representations of words.

- These vectors capture semantic meanings by placing words with similar contexts closer together in a vector space. Unlike sparse representations,
- Word2Vec embedding's have fewer zero values because they are dense, meaning they store information efficiently.
- They typically have low dimensions (e.g., 100 to 300), making them computationally efficient while preserving meaningful relationships between words.

	King	Queen	Man	Women	Monkey
Gender	1	1	1	1	1
Wealth	1	1	0.3	0.3	0
Power	1	0.7	0.2	0.2	0
Weight	0.8	0.4	0.6	0.5	0.3
Speak	1	1	1	1	0

King	Man	Women	King - Man + Women = Queen
1	1	1	1 - 1 + 1 = 1
1	0.3	0.3	1 - 0.3 + 0.3 = 1
1	0.4	0.2	1 - 0.4 + 0.2 = 0.8
0.8	0.6	0.3	0.8 - 0.6 + 0.3 = 0.5
1	1	1	1 - 1 + 1 = 1

Queen	Women	Man	Queen - Women + Man = King
1	1	1	1 - 1 + 1 = 1
1	0.3	0.3	1 - 0.3 + 0.3 = 1
0.7	0.2	0.2	0.7 - 0.2 + 0.2 = 0.7
0.4	0.5	0.6	0.4 - 0.5 + 0.6 = 0.5
1	1	1	1 - 1 + 1 = 1

Type of Word2Vec

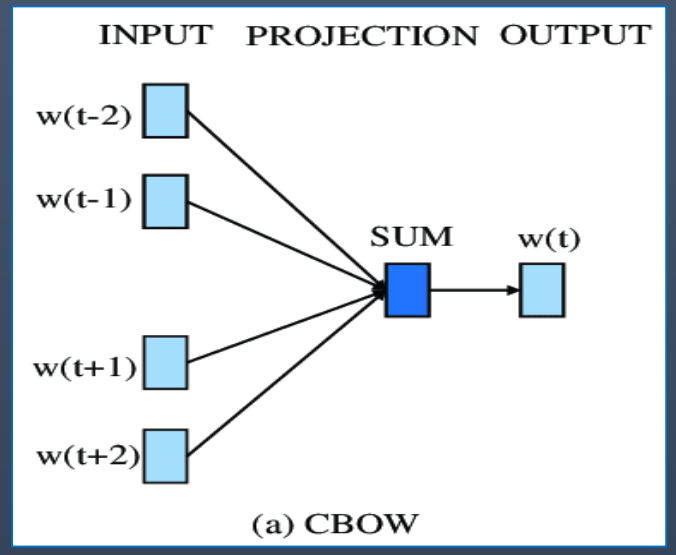
1) **Continuous Bag of Words (CBOW):**
2) **Skip-gram:**

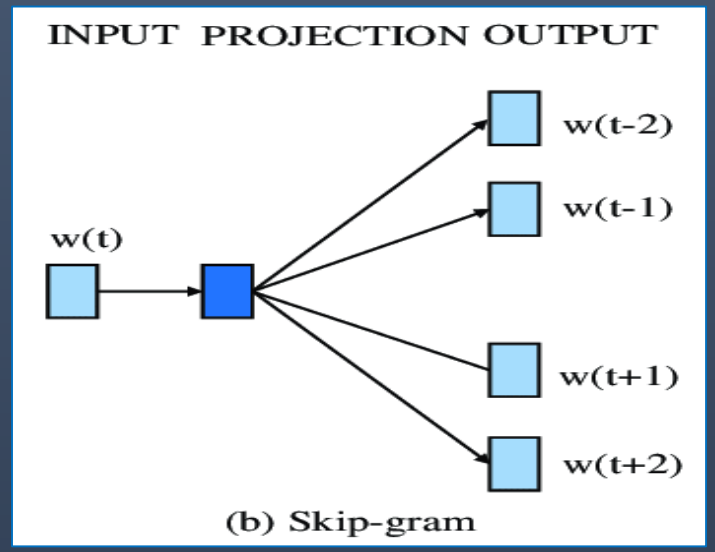
Continuous Bag of Words (CBOW): Predicts a **missing/target word using the surrounding words.**

Skip-gram: Skip-Gram predicts the surrounding/context words from a given target word.

Slide 1 Watch _____ for Data Science

Slide 1 _____ YouTube ___ Data Science





D1 Watch YouTube for Data Science

	Watch	YouTube	for	Data	Science
Watch	1	0	0	0	0
YouTube	0	1	0	0	0
for	0	0	1	0	0
Data	0	0	0	1	0
Science	0	0	0	0	1

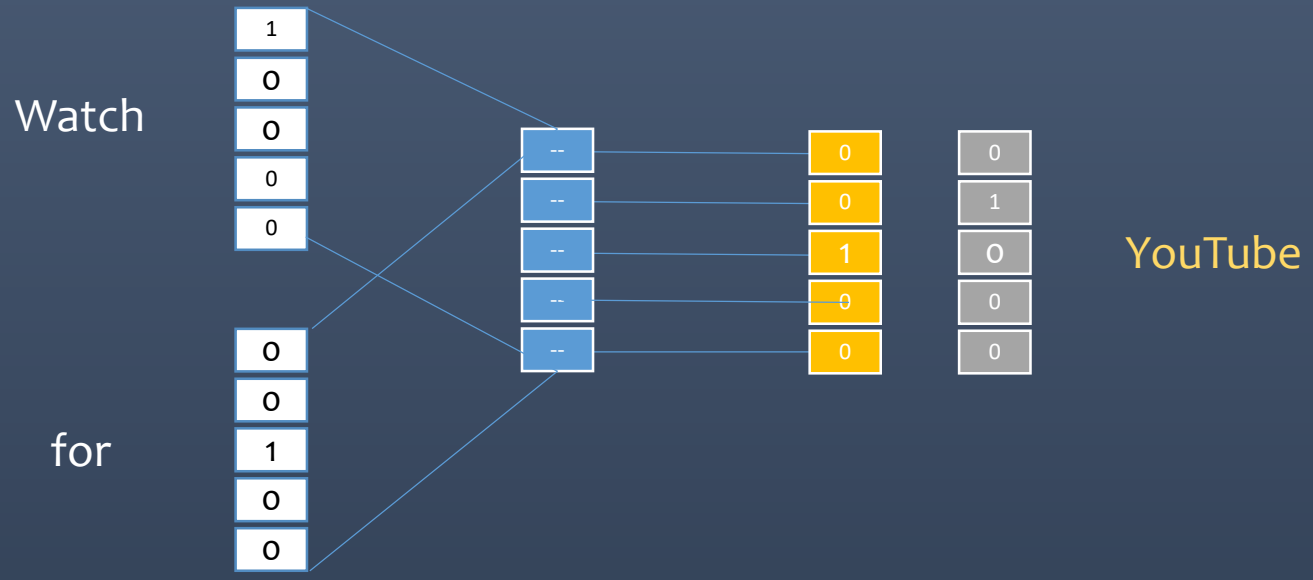
	King	Queen	Man	Women	Monkey
Gender	1	1	1	1	1
Wealth	1	1	0.3	0.3	0
Power	1	0.7	0.2	0.2	0
Weight	0.8	0.4	0.6	0.5	0.3
Speak	1	1	1	1	0

Slide 1 Watch YouTube for Data Science

Slide 2 Watch YouTube for Data Science

Slide 3 Watch YouTube for Data Science

	X1	X2	Y
Slide 1	Watch	for	YouTube
Slide 2	YouTube	Data	for
Slide 3	for	Science	Data



D1 Watch YouTube for Data Science

	Watch	YouTube	for	Data	Science
Watch	1	0	0	0	0
YouTube	0	1	0	0	0
for	0	0	1	0	0
Data	0	0	0	1	0
Science	0	0	0	0	1

	King	Queen	Man	Women	Monkey
Gender	1	1	1	1	1
Wealth	1	1	0.3	0.3	0
Power	1	0.7	0.2	0.2	0
Weight	0.8	0.4	0.6	0.5	0.3
Speak	1	1	1	1	0

Slide 1 Watch YouTube for Data Science

Slide 2 Watch YouTube for Data Science

Slide 3 Watch YouTube for Data Science

	x	Y1	Y2
Slide 1	YouTube	Watch	For
Slide 2	for	YouTube	Data
Slide 3	Data	for	Science

